

eBike Rental System

Software Test Plan

Field	Details
Project Name	IT342 eBike Rental System
Version	1.0
Build Number	Phase 3 - Vertical Slice Refactored
Test Date(s)	May 11-12, 2026
Tested By	QA Team
Backend	Spring Boot 3.x, Java 21, PostgreSQL (Neon Cloud)
Frontend	React 18, TypeScript, Vite
Mobile	Android (Kotlin), Compose UI
Database	PostgreSQL with SSL
Ports	Backend 8083 Frontend 5173 Mobile (Device)

PART 1: FUNCTIONAL REQUIREMENTS

1.1 Authentication & User Management (FR-001 to FR-010)

FR ID	Requirement	Module	Priority
FR-001	User can register with email and password	Auth	High
FR-002	User can log in with email and password	Auth	High
FR-003	User can log in via Google OAuth	Auth	Medium
FR-004	JWT token is generated on successful login	Auth	High
FR-005	JWT token validates on protected routes	Auth	High
FR-006	User can view their profile information	User	Medium
FR-007	User can update their profile (name, phone, address)	User	Medium
FR-008	User can upload profile picture	User	Low
FR-009	User can log out	Auth	High
FR-010	User password is securely hashed (BCrypt)	Auth	High

1.2 Bike Management (FR-011 to FR-020)

FR ID	Requirement	Module	Priority
FR-011	Users can view all available bikes	Bike	High
FR-012	Users can search bikes by type/brand/location	Bike	Medium
FR-013	Users can view bike details (specs, price, condition)	Bike	High
FR-014	Bike availability status is displayed accurately	Bike	High
FR-015	Bike battery level is displayed correctly	Bike	Medium
FR-016	Bike pricing (hourly/daily) is calculated correctly	Bike	High
FR-017	Admin can create new bikes	Bike	High
FR-018	Admin can update bike information	Bike	High
FR-019	Admin can delete bikes from inventory	Bike	Medium
FR-020	Bikes are filtered by status (AVAILABLE, RENTED, MAINTENANCE)	Bike	High

1.3 Booking Management (FR-021 to FR-030)

FR ID	Requirement	Module	Priority
FR-021	User can create a booking for available bike	Booking	High
FR-022	Booking displays correct rental duration	Booking	High
FR-023	System prevents double-booking same bike	Booking	High
FR-024	Booking price is calculated accurately	Booking	High
FR-025	User can view booking history	Booking	Medium
FR-026	User can cancel booking (before start time)	Booking	High
FR-027	User receives booking confirmation	Booking	Medium
FR-028	Admin can view all active rentals	Booking	High
FR-029	Admin can complete bookings	Booking	High
FR-030	Bike status auto-updates when booking starts/ends	Booking	High

1.4 Payment Processing (FR-031 to FR-040)

FR ID	Requirement	Module	Priority
FR-031	User can pay via Stripe (credit/debit card)	Payment	High
FR-032	User can pay via GCash (cash payment method)	Payment	High
FR-033	Payment processing completes successfully	Payment	High
FR-034	Transaction ID is generated for each payment	Payment	High
FR-035	Payment history is tracked and logged	Payment	Medium
FR-036	User receives payment confirmation	Payment	Medium
FR-037	Failed payments are handled gracefully	Payment	High
FR-038	Refunds can be processed by admin	Payment	Medium
FR-039	Payment status is updated in real-time	Payment	High
FR-040	Multiple payment methods are supported	Payment	Medium

1.5 Admin Dashboard (FR-041 to FR-050)

FR ID	Requirement	Module	Priority
FR-041	Admin can view all users	Admin	Medium
FR-042	Admin can view all bookings	Admin	High
FR-043	Admin can view active rentals	Admin	High
FR-044	Admin can view system statistics/reports	Admin	Low
FR-045	Admin can manage bike inventory	Admin	High
FR-046	Admin can manage user accounts	Admin	Medium
FR-047	Admin can process refunds	Admin	Medium
FR-048	Admin dashboard shows revenue reports	Admin	Low
FR-049	Admin can search bookings by date range	Admin	Medium
FR-050	Admin has exclusive access (role-based)	Admin	High

1.6 API & Backend (FR-051 to FR-060)

FR ID	Requirement	Module	Priority
FR-051	API endpoints return proper JSON responses	API	High
FR-052	Error responses include descriptive messages	API	High
FR-053	API validates input data on all endpoints	API	High
FR-054	CORS is configured for frontend access	API	High

FR ID	Requirement	Module	Priority
FR-055	API rate limiting is implemented	API	Low
FR-056	Database connection pooling is active	API	Medium
FR-057	API logs all operations for audit trail	API	Medium
FR-058	API is secured against SQL injection	API	High
FR-059	API timeout and error handling works	API	High
FR-060	Health check endpoints return correct status	API	Medium

PART 2: REQUIREMENTS TRACEABILITY MATRIX (RTM)

FR ID	Requirement	TC ID	Manual	Automated	Status
FR-001	User can register with email and password	TC-001	Yes	Yes	Covered
FR-002	User can log in with email and password	TC-002	Yes	Yes	Covered
FR-003	User can log in via Google OAuth	TC-003	Yes	Yes	Covered
FR-004	JWT token is generated on successful login	TC-004	Yes	Yes	Covered
FR-005	JWT token validates on protected routes	TC-005	Yes	Yes	Covered
FR-006	User can view their profile information	TC-006	Yes	Yes	Covered
FR-007	User can update their profile (name, phone, address)	TC-007	Yes	Yes	Covered
FR-008	User can upload profile picture	TC-008	Yes	Yes	Covered
FR-009	User can log out	TC-009	Yes	Yes	Covered
FR-010	User password is securely hashed (BCrypt)	TC-010	Yes	Yes	Covered
FR-011	Users can view all available bikes	TC-011	Yes	Yes	Covered
FR-012	Users can search bikes by type/brand/location	TC-012	Yes	Yes	Covered
FR-013	Users can view bike details (specs, price, condition)	TC-013	Yes	Yes	Covered
FR-014	Bike availability status is displayed accurately	TC-014	Yes	Yes	Covered
FR-015	Bike battery level is displayed correctly	TC-015	Yes	Yes	Covered
FR-016	Bike pricing (hourly/daily) is calculated correctly	TC-016	Yes	Yes	Covered

FR ID	Requirement	TC ID	Manual	Automated	Status
FR-017	Admin can create new bikes	TC-017	Yes	Yes	Covered
FR-018	Admin can update bike information	TC-018	Yes	Yes	Covered
FR-019	Admin can delete bikes from inventory	TC-019	Yes	Yes	Covered
FR-020	Bikes are filtered by status (AVAILABLE, RENTED, MAINTENANCE)	TC-020	Yes	Yes	Covered
FR-021	User can create a booking for available bike	TC-021	Yes	Yes	Covered
FR-022	Booking displays correct rental duration	TC-022	Yes	Yes	Covered
FR-023	System prevents double-booking same bike	TC-023	Yes	Yes	Covered
FR-024	Booking price is calculated accurately	TC-024	Yes	Yes	Covered
FR-025	User can view booking history	TC-025	Yes	Yes	Covered
FR-026	User can cancel booking (before start time)	TC-026	Yes	Yes	Covered
FR-027	User receives booking confirmation	TC-027	Yes	Yes	Covered
FR-028	Admin can view all active rentals	TC-028	Yes	Yes	Covered
FR-029	Admin can complete bookings	TC-029	Yes	Yes	Covered
FR-030	Bike status auto-updates when booking starts/ends	TC-030	Yes	Yes	Covered
FR-031	User can pay via Stripe (credit/debit card)	TC-031	Yes	Yes	Covered
FR-032	User can pay via GCash (cash payment method)	TC-032	Yes	Yes	Covered
FR-033	Payment processing completes successfully	TC-033	Yes	Yes	Covered
FR-034	Transaction ID is generated for each payment	TC-034	Yes	Yes	Covered
FR-035	Payment history is tracked and logged	TC-035	Yes	Yes	Covered
FR-036	User receives payment confirmation	TC-036	Yes	Yes	Covered
FR-037	Failed payments are handled gracefully	TC-037	Yes	Yes	Covered
FR-038	Refunds can be processed by admin	TC-038	Yes	Yes	Covered
FR-039	Payment status is updated in real-time	TC-039	Yes	Yes	Covered
FR-040	Multiple payment methods are supported	TC-040	Yes	Yes	Covered
FR-041	Admin can view all users	TC-041	Yes	Yes	Covered
FR-042	Admin can view all bookings	TC-042	Yes	Yes	Covered

FR ID	Requirement	TC ID	Manual	Automated	Status
FR-043	Admin can view active rentals	TC-043	Yes	Yes	Covered
FR-044	Admin can view system statistics/reports	TC-044	Yes	Yes	Covered
FR-045	Admin can manage bike inventory	TC-045	Yes	Yes	Covered
FR-046	Admin can manage user accounts	TC-046	Yes	Yes	Covered
FR-047	Admin can process refunds	TC-047	Yes	Yes	Covered
FR-048	Admin dashboard shows revenue reports	TC-048	Yes	Pending	Covered
FR-049	Admin can search bookings by date range	TC-049	Yes	Yes	Covered
FR-050	Admin has exclusive access (role-based)	TC-050	Yes	Yes	Covered
FR-051	API endpoints return proper JSON responses	TC-051	Yes	Yes	Covered
FR-052	Error responses include descriptive messages	TC-052	Yes	Yes	Covered
FR-053	API validates input data on all endpoints	TC-053	Yes	Yes	Covered
FR-054	CORS is configured for frontend access	TC-054	Yes	Yes	Covered
FR-055	API rate limiting is implemented	TC-055	Yes	Yes	Covered
FR-056	Database connection pooling is active	TC-056	Yes	Yes	Covered
FR-057	API logs all operations for audit trail	TC-057	Yes	Yes	Covered
FR-058	API is secured against SQL injection	TC-058	Yes	Yes	Covered
FR-059	API timeout and error handling works	TC-059	Yes	Yes	Covered
FR-060	Health check endpoints return correct status	TC-060	Yes	Yes	Covered

Total Requirements: 60 | Total Test Cases: 60 | Coverage: 100%

PART 3: TEST CASES — DETAILED FORMAT

TC-001: Valid User Registration

Field	Value
Test Case ID	TC-001

Field	Value
Test Case Name	Valid User Registration
Requirement ID	FR-001
Preconditions	Backend API running on http://localhost:8083/api; Frontend running on http://localhost:5173; No user exists with email testuser@example.com
Test Steps	1. Navigate to registration page at http://localhost:5173/register2. Enter first name: 'John'3. Enter last name: 'Doe'4. Enter email: 'testuser@example.com'5. Enter password: 'Test@1234'6. Enter phone: '09123456789'7. Click 'Register' button
Expected Result	1. Response status 201 Created2. User account created in database3. Redirect to login page4. Success message displayed
Actual Result	[PENDING]
Status	[PENDING]

TC-002: Valid User Login

Field	Value
Test Case ID	TC-002
Test Case Name	Valid User Login
Requirement ID	FR-002
Preconditions	Backend running; Registered user exists (email: testuser@example.com)
Test Steps	1. Navigate to http://localhost:5173/login2. Enter email: 'testuser@example.com'3. Enter password: 'Test@1234'4. Click 'Login' button
Expected Result	1. HTTP 200 OK2. JWT token returned3. Redirected to dashboard4. User name displayed
Actual Result	[PENDING]
Status	[PENDING]

TC-004: JWT Token Generated on Login

Field	Value
Test Case ID	TC-004
Test Case Name	JWT Token Generated on Login
Requirement ID	FR-004
Preconditions	Backend running on http://localhost:8083; Valid user exists

Field	Value
Test Steps	1. POST /auth/login with valid credentials2. Capture response body
Expected Result	1. Response status 2002. Response contains JWT token in format: 'eyJhbGc...'3. Token contains user id and roles4. Token expiration is set
Actual Result	[PENDING]
Status	[PENDING]

TC-005: JWT Token Validates Protected Routes

Field	Value
Test Case ID	TC-005
Test Case Name	JWT Token Validates Protected Routes
Requirement ID	FR-005
Preconditions	Valid JWT token from login; Backend running
Test Steps	1. GET /bookings with Authorization: Bearer {valid_token}2. GET /bookings without Authorization header3. GET /bookings with invalid token
Expected Result	1. With valid token: HTTP 200, bookings returned2. Without token: HTTP 401 Unauthorized3. With invalid token: HTTP 401 Unauthorized
Actual Result	[PENDING]
Status	[PENDING]

PART 4: MANUAL TEST SCRIPTS

Test Script: TC-001 — User Registration

```
Feature: User Registration
  Scenario: User successfully registers with valid email and password
    Given I am on the registration page
    And I enter email "newuser@example.com"
    And I enter password "Secure@Pass123"
    And I enter first name "Jane"
    And I enter last name "Smith"
    And I enter phone "09123456789"
    When I click the "Register" button
    Then I should see "Registration successful"
    And I should be redirected to the login page
```

Test Script: TC-002 — User Login


```
Feature: User Login
  Scenario: User successfully logs in with valid credentials
    Given I am on the login page
    And I enter email "testuser@example.com"
    And I enter password "Test@1234"
    When I click the "Login" button
    Then I should see my dashboard
    And my user name should be displayed
    And I should see available bikes
```

Test Script: TC-011 — View All Bikes

```
Feature: View Bikes
  Scenario: User views list of available bikes
    Given I am logged in
    And I am on the bikes page
    When the page loads
    Then I should see a list of available bikes
    And Each bike should display: Bike Code, Model, Price, Battery, Status
```

PART 5: AUTOMATED TEST CASES

5.1 Backend API Tests (Java/JUnit)

```
@SpringBootTest
public class BikeServiceTest {
    @Autowired private BikeService bikeService;
    @Autowired private BikeRepository bikeRepository;

    @Test
    public void testGetAllBikes() {
        List<Bike> bikes = bikeService.getAllBikes();
        assertNotNull(bikes);
        assertFalse(bikes.isEmpty());
    }

    @Test
    public void testCreateBike() {
        Bike newBike = new Bike();
        newBike.setBikeCode("TEST001");
        newBike.setModel("Test Bike");
        newBike.setPricePerHour(10.0);
        Bike created = bikeService.createBike(newBike);
        assertNotNull(created.getId());
        assertEquals("TEST001", created.getBikeCode());
    }
}
```

5.2 Frontend Tests (React/Jest)

```
describe('BikeList Component', () => {
  test('renders bike list on load', async () => {
    render(<BikeList />);
    await waitFor(() => {
      expect(screen.getByText(/Mountain Pro X/i)).toBeInTheDocument();
    });
  });
  test('displays bike details correctly', async () => {
    render(<BikeList />);
    await waitFor(() => {
      const bikes = screen.getAllByRole('article');
      expect(bikes.length).toBeGreaterThan(0);
    });
  });
});
```

5.3 Mobile Tests (Kotlin/Compose)

```
@RunWith(AndroidUnit4::class)
class BikeListScreenTest {
    @get:Rule
    val composeTestRule = createComposeRule()

    @Test
    fun bikeListDisplaysCorrectly() {
        composeTestRule.setContent { BikeListScreen() }
        composeTestRule.onNodeWithText("Mountain Pro X").assertIsDisplayed()
        composeTestRule.onNodeWithText("5.00/hour").assertIsDisplayed()
    }
}
```

SUMMARY

Metric	Value
Total Functional Requirements	60
Total Test Cases	60
Manual Test Coverage	100%
Automated Test Coverage	90%
Overall Coverage	100%